

ΓΚ802 - Συνεδρίες και διαχείριση πρόσβασης χρηστών

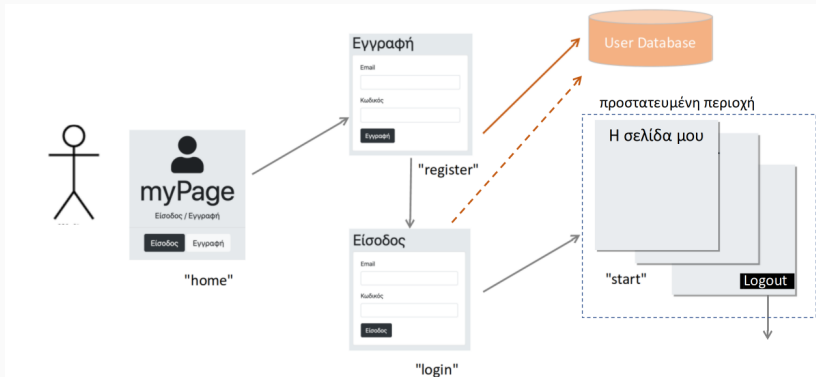
Ν. Αβούρης - Χ. Φείδας - Χ. Σιντόρης

2024

ECE/UPatras

Συνεδρίες και διαχείριση πρόσβασης χρηστών

Συνεδρίες και διαχείριση πρόσβασης χρηστών



Σχήμα 1: Κάποια από τα περιεχόμενα της ιστοσελίδας μπορεί να βρίσκονται σε προστατευμένη περιοχή. Μόνο χρήστες που έχουν αυθεντικοποιηθεί μπορούν να έχουν πρόσβαση.

Απαιτήσεις από το σύστημα διαχείρισης χρηστών

Βασικές απαιτήσεις από το σύστημά μας είναι:

1. Να μην επιτρέπει σε **μη αυθεντικοποιημένους** χρήστες να έχουν πρόσβαση στο **προστατευμένο τμήμα** της εφαρμογής,

Απαιτήσεις από το σύστημα διαχείρισης χρηστών

Βασικές απαιτήσεις από το σύστημά μας είναι:

1. Να μην επιτρέπει σε **μη αυθεντικοποιημένους** χρήστες να έχουν πρόσβαση στο **προστατευμένο τμήμα** της εφαρμογής,
2. να προστατεύει τα **διαπιστευτήρια** (credentials) των χρηστών ώστε να είναι προστατευμένοι *ακόμη και αν αυτά πέσουν στα χέρια τρίτων*,

Απαιτήσεις από το σύστημα διαχείρισης χρηστών

Βασικές απαιτήσεις από το σύστημά μας είναι:

1. Να μην επιτρέπει σε **μη αυθεντικοποιημένους** χρήστες να έχουν πρόσβαση στο **προστατευμένο τμήμα** της εφαρμογής,
2. να προστατεύει τα **διαπιστευτήρια** (credentials) των χρηστών ώστε να είναι προστατευμένοι *ακόμη και αν αυτά πέσουν στα χέρια τρίτων*,
3. να παρέχει μηχανισμούς ώστε, για ένα εύλογο διάστημα, να επιτρέπεται στους χρήστες που έχουν αναγνωριστεί να επιστρέφουν στην προστατευμένη περιοχή χωρίς να χρειάζεται να περάσουν από διαδικασία αυθεντικοποίησης (**διαχείριση συνεδρίας** - session management).

Απαιτήσεις από το σύστημα διαχείρισης χρηστών

Βασικές απαιτήσεις από το σύστημά μας είναι:

1. Να μην επιτρέπει σε **μη αυθεντικοποιημένους** χρήστες να έχουν πρόσβαση στο **προστατευμένο τμήμα** της εφαρμογής,
 2. να προστατεύει τα **διαπιστευτήρια** (credentials) των χρηστών ώστε να είναι προστατευμένοι *ακόμη και αν αυτά πέσουν στα χέρια τρίτων*,
 3. να παρέχει μηχανισμούς ώστε, για ένα εύλογο διάστημα, να επιτρέπεται στους χρήστες που έχουν αναγνωριστεί να επιστρέφουν στην προστατευμένη περιοχή χωρίς να χρειάζεται να περάσουν από διαδικασία αυθεντικοποίησης (**διαχείριση συνεδρίας** - session management).
- Ποιο είναι ένα εύλογο χρονικό διάστημα;

1. Πρόσβαση στην προστατευμένη περιοχή

Αυθεντικοποίηση χρηστών

Η ιδέα της αυθεντικοποίησης (authentication) χρηστών είναι απλή:
Να αναγνωρίσει το σύστημα τον συγκεκριμένο χρήστη ώστε να του επιτρέψει την πρόσβαση σε κάποιες από τις προστατευμένες περιοχές του.

Η αναγνώριση αυτού του δικαιώματος μπορεί να γίνει

- με κάτι που ο χρήστης αποδεικνύει πως γνωρίζει (συνθηματικό, pin κλπ),

Αυθεντικοποίηση χρηστών

Η ιδέα της αυθεντικοποίησης (authentication) χρηστών είναι απλή: Να αναγνωρίσει το σύστημα τον συγκεκριμένο χρήστη ώστε να του επιτρέψει την πρόσβαση σε κάποιες από τις προστατευμένες περιοχές του.

Η αναγνώριση αυτού του δικαιώματος μπορεί να γίνει

- με κάτι που ο χρήστης αποδεικνύει πως γνωρίζει (συνθηματικό, pin κλπ),
- με κάτι που ο χρήστης αποδεικνύει πως κατέχει (ψηφ. πιστοποιητικό, κιν. τηλέφωνο κλπ),

Αυθεντικοποίηση χρηστών

Η ιδέα της αυθεντικοποίησης (authentication) χρηστών είναι απλή: Να αναγνωρίσει το σύστημα τον συγκεκριμένο χρήστη ώστε να του επιτρέψει την πρόσβαση σε κάποιες από τις προστατευμένες περιοχές του.

Η αναγνώριση αυτού του δικαιώματος μπορεί να γίνει

- με κάτι που ο χρήστης αποδεικνύει πως γνωρίζει (συνθηματικό, pin κλπ),
- με κάτι που ο χρήστης αποδεικνύει πως κατέχει (ψηφ. πιστοποιητικό, κιν. τηλέφωνο κλπ),
- έμμεσα (βιομετρικά χαρακτηριστικά, συμπεριφορά κλπ).

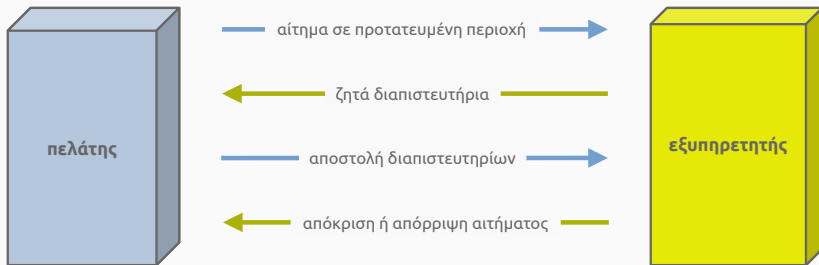
Αυθεντικοποίηση χρηστών

Η ιδέα της αυθεντικοποίησης (authentication) χρηστών είναι απλή: Να αναγνωρίσει το σύστημα τον συγκεκριμένο χρήστη ώστε να του επιτρέψει την πρόσβαση σε κάποιες από τις προστατευμένες περιοχές του.

Η αναγνώριση αυτού του δικαιώματος μπορεί να γίνει

- με κάτι που ο χρήστης αποδεικνύει πως γνωρίζει (συνθηματικό, pin κλπ),
- με κάτι που ο χρήστης αποδεικνύει πως κατέχει (ψηφ. πιστοποιητικό, κιν. τηλέφωνο κλπ),
- έμμεσα (βιομετρικά χαρακτηριστικά, συμπεριφορά κλπ).
- Η αυθεντικοποίηση μπορεί να συνδυάζει μεθόδους (multi-factor authentication)

Πρόσβαση με όνομα χρήστη και συνθηματικό

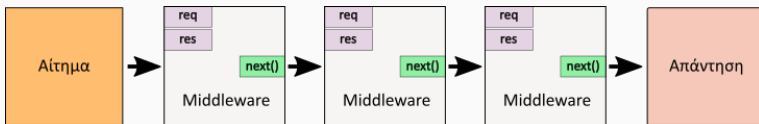


Σχήμα 2: Αυθεντικοποίηση χρήστη χρησιμοποιώντας κάποιο είδος διαπιστευτηρίων.

Προστασία περιοχής με το Express.js

Σε κάθε αίτημα στον εξυπηρετητή που αφορά προστατευμένη περιοχή, πριν ικανοποιηθεί το αίτημα γίνεται έλεγχος αν ο χρήστης είναι αυθεντικοποιημένος.

- Εκμεταλλευόμαστε το μηχανισμό middleware του Express.js:

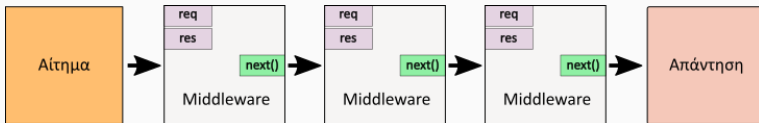


Σχήμα 3: Η αλυσίδα επεξεργασίας του Express.js.

Προστασία περιοχής με το Express.js

Σε κάθε αίτημα στον εξυπηρετητή που αφορά προστατευμένη περιοχή, πριν ικανοποιηθεί το αίτημα γίνεται έλεγχος αν ο χρήστης είναι αυθεντικοποιημένος.

- Εκμεταλλευόμαστε το μηχανισμό middleware του Express.js:



Σχήμα 3: Η αλυσίδα επεξεργασίας του Express.js.

- Για να προστατέψουμε την πρόσβαση σε περιοχές, το 1ο υποτμήμα της αλυσίδας επεξεργασίας ελέγχει αν ο χρήστης έχει δικαίωμα πρόσβασης, π.χ. αν είναι αυθεντικοποιημένος.

Προστασία περιοχής με το Express.js 2

Σε κάθε αίτημα στον εξυπηρετητή που αφορά προστατευμένη περιοχή, πριν ικανοποιηθεί το αίτημα γίνεται έλεγχος αν ο χρήστης είναι αυθεντικοποιημένος.

- Εκμεταλλευόμαστε το μηχανισμό middleware του Express.js:

```
router.get('/tasks/remove/:removeTaskId',  
  loginController.checkAuthenticated,  
  taskListController.removeTask);
```


Προστασία περιοχής με το Express.js 2

Σε κάθε αίτημα στον εξυπηρετητή που αφορά προστατευμένη περιοχή, πριν ικανοποιηθεί το αίτημα γίνεται έλεγχος αν ο χρήστης είναι αυθεντικοποιημένος.

- Εκμεταλλευόμαστε το μηχανισμό middleware του Express.js:

```
router.get('/tasks/remove/:removeTaskId',  
  loginController.checkAuthenticated,  
  taskListController.removeTask);
```

- όπου, σε άλλο αρχείο (module):

```
export function checkAuthenticated(req, res, next) {  
  if (authenticated) { next(); }  
  else {  
    res.redirect('/login'); }  
}
```

2. Προστασία των διαπιστευτηρίων

Τα διαπιστευτήρια του χρήστη χρειάζεται να είναι διαθέσιμα στο σύστημα, ώστε να μπορούν κάθε φορά να συγκριθούν με αυτά που θα παρέχει ο χρήστης που ζητά πρόσβαση.

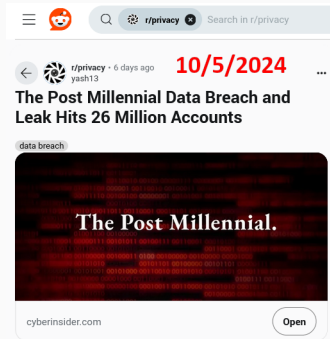
- Ο πιο απλός τρόπος είναι η αποθήκευση του ζεύγους **όνομα χρήστη/συνθηματικό** στη ΒΔ του συστήματος.

Φύλαξη των διαπιστευτηρίων

Τα διαπιστευτήρια του χρήστη χρειάζεται να είναι διαθέσιμα στο σύστημα, ώστε να μπορούν κάθε φορά να συγκριθούν με αυτά που θα παρέχει ο χρήστης που ζητά πρόσβαση.

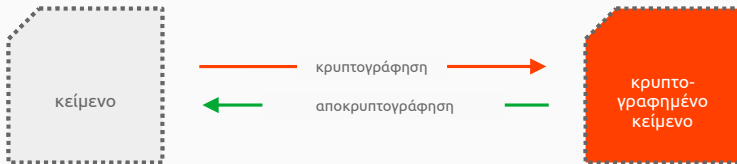
- Ο πιο απλός τρόπος είναι η αποθήκευση του ζεύγους **όνομα χρήστη/συνθηματικό** στη ΒΔ του συστήματος.
- Στην απλούστερη και λιγότερο ασφαλή της εκδοχή είναι στη μορφή **μη κρυπτογραφημένου κειμένου** (cleartext).

```
{ "_id" : "1", "email" : "a@a.com", "password" : "123" }
```

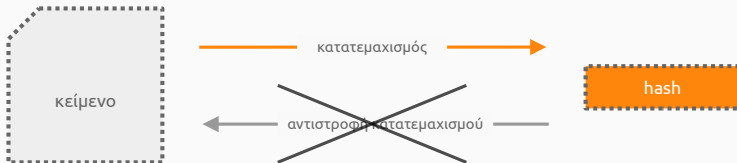


Σχήμα 4: The Post Millennial Data Breach and Leak Hits 26 Million Accounts - 10/5/2024

Προστασία διαπιστευτηρίων με κρυπτογράφηση



Σχήμα 5: Η κρυπτογράφηση ενός μηνύματος παράγει κρυπτογραφημένο μήνυμα ανάλογου μεγέθους με το αρχικό.



Σχήμα 6: Ο μονόδρομος κατατεμαχισμός (one-way hash) παράγει περίληψη (hash) σταθερού μήκους.

Κατατεμαχισμός - Hashing

Μια συνάρτηση κατατεμαχισμού, $H(x)$, παράγει περίληψη (“**digest**” ή “**hash**”) ενός κειμένου οποιουδήποτε μεγέθους:

- εύκολα και γρήγορα,

Κατατεμαχισμός - Hashing

Μια συνάρτηση κατατεμαχισμού, $H(x)$, παράγει περίληψη (“digest” ή “hash”) ενός κειμένου οποιουδήποτε μεγέθους:

- εύκολα και γρήγορα,
- περίληψη **ίδιου** μεγέθους για οποιοδήποτε μέγεθος κειμένου,

Κατατεμαχισμός - Hashing

Μια συνάρτηση κατατεμαχισμού, $H(x)$, παράγει περίληψη (“**digest**” ή “**hash**”) ενός κειμένου οποιουδήποτε μεγέθους:

- εύκολα και γρήγορα,
- περίληψη **ίδιου** μεγέθους για οποιοδήποτε μέγεθος κειμένου,
- για το ίδιο κείμενο παράγεται **πάντα** η **ίδια** περίληψη,

Κατατεμαχισμός - Hashing

Μια συνάρτηση κατατεμαχισμού, $H(x)$, παράγει περίληψη (“digest” ή “hash”) ενός κειμένου οποιουδήποτε μεγέθους:

- εύκολα και γρήγορα,
- περίληψη **ίδιου** μεγέθους για οποιοδήποτε μέγεθος κειμένου,
- για το ίδιο κείμενο παράγεται **πάντα** η **ίδια** περίληψη,
- είναι **υπολογιστικά ανέφικτο**:

Κατατεμαχισμός - Hashing

Μια συνάρτηση κατατεμαχισμού, $H(x)$, παράγει περίληψη (“digest” ή “hash”) ενός κειμένου οποιουδήποτε μεγέθους:

- εύκολα και γρήγορα,
- περίληψη **ίδιου** μεγέθους για οποιοδήποτε μέγεθος κειμένου,
- για το ίδιο κείμενο παράγεται **πάντα** η **ίδια** περίληψη,
- είναι **υπολογιστικά ανέφικτο**:
 - για οποιαδήποτε περίληψη d , να αποκαλυφθεί μήνυμα x για το οποίο να ισχύει $H(x) = d$ (*preimage resistance*),

Κατατεμαχισμός - Hashing

Μια συνάρτηση κατατεμαχισμού, $H(x)$, παράγει περίληψη (“**digest**” ή “**hash**”) ενός κειμένου οποιουδήποτε μεγέθους:

- εύκολα και γρήγορα,
- περίληψη **ίδιου** μεγέθους για οποιοδήποτε μέγεθος κειμένου,
- για το ίδιο κείμενο παράγεται **πάντα** η **ίδια** περίληψη,
- είναι **υπολογιστικά ανέφικτο**:
 - για οποιαδήποτε περίληψη d , να αποκαλυφθεί μήνυμα x για το οποίο να ισχύει $H(x) = d$ (*preimage resistance*),
 - για οποιοδήποτε μήνυμα x , να βρεθεί $y \neq x$ για το οποίο ισχύει $H(x) = H(y) = d$ (*2nd preimage resistance*),

Κατατεμαχισμός - Hashing

Μια συνάρτηση κατατεμαχισμού, $H(x)$, παράγει περίληψη (“**digest**” ή “**hash**”) ενός κειμένου οποιουδήποτε μεγέθους:

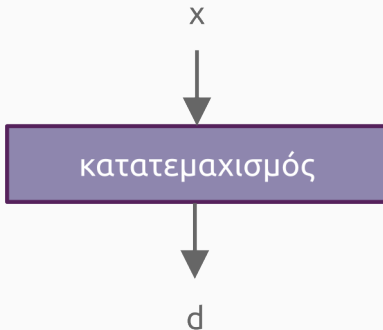
- εύκολα και γρήγορα,
- περίληψη **ίδιου** μεγέθους για οποιοδήποτε μέγεθος κειμένου,
- για το ίδιο κείμενο παράγεται **πάντα** η **ίδια** περίληψη,
- είναι **υπολογιστικά ανέφικτο**:
 - για οποιαδήποτε περίληψη d , να αποκαλυφθεί μήνυμα x για το οποίο να ισχύει $H(x) = d$ (*preimage resistance*),
 - για οποιοδήποτε μήνυμα x , να βρεθεί $y \neq x$ για το οποίο ισχύει $H(x) = H(y) = d$ (*2nd preimage resistance*),
 - να βρεθεί ζεύγος (x, y) για το οποίο να ισχύει: $H(x) = H(y)$ (*collision resistance*).

Κατατεμαχισμός - Hashing

Μια συνάρτηση κατατεμαχισμού, $H(x)$, παράγει περίληψη (“digest” ή “hash”) ενός κειμένου οποιουδήποτε μεγέθους:

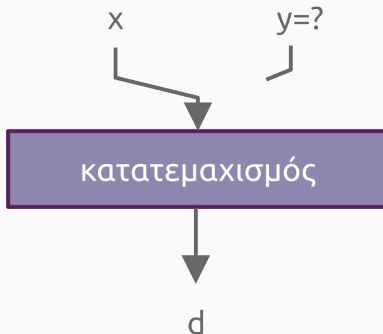
- εύκολα και γρήγορα,
- περίληψη **ίδιου** μεγέθους για οποιοδήποτε μέγεθος κειμένου,
- για το ίδιο κείμενο παράγεται **πάντα** η **ίδια** περίληψη,
- είναι **υπολογιστικά ανέφικτο**:
 - για οποιαδήποτε περίληψη d , να αποκαλυφθεί μήνυμα x για το οποίο να ισχύει $H(x) = d$ (*preimage resistance*),
 - για οποιοδήποτε μήνυμα x , να βρεθεί $y \neq x$ για το οποίο ισχύει $H(x) = H(y) = d$ (*2nd preimage resistance*),
 - να βρεθεί ζεύγος (x, y) για το οποίο να ισχύει: $H(x) = H(y)$ (*collision resistance*).
 - “Preimage attack” = Επίθεση αντίστροφης εικόνας, η απόπειρα να βρεθεί μήνυμα που να οδηγεί σε συγκεκριμένη περίληψη

Κατατεμαχισμός - preimage resistance



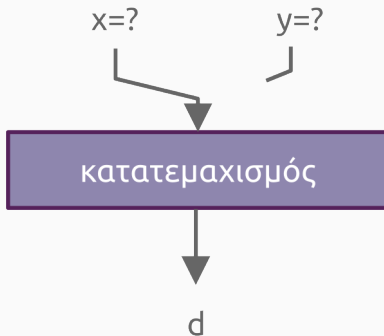
Σχήμα 7: Να είναι **υπολογιστικά ανέφικτο** για οποιαδήποτε περίληψη **d**, να αποκαλυφθεί μήνυμα **x** για το οποίο να ισχύει **$H(x) = d$** (*preimage resistance*).

Κατατεμαχισμός - 2nd preimage resistance



Σχήμα 8: Να είναι **υπολογιστικά ανέφικτο** για οποιοδήποτε μήνυμα x , να βρεθεί $y \neq x$ για το οποίο ισχύει $H(x) = H(y) = d$ (2nd preimage resistance).

Κατατεμαχισμός - collision resistance



Σχήμα 9: Να είναι **υπολογιστικά ανέφικτο** να βρεθεί ζεύγος (x, y) για το οποίο να ισχύει: $H(x) = H(y)$ (*collision resistance*).

Αποθήκευση της περίληψης του κατατεμαχισμού



Σχήμα 10: Η συνάρτηση κατατεμαχισμού μας δίνει τη δυνατότητα να αποθηκεύσουμε την **περίληψη** και όχι το το συνθηματικό (δηλ. το αρχικό μήνυμα). Από τη βάση δεδομένων μπορεί πλέον να υποκλαπεί μόνο η περίληψη.

Κατατεμαχισμός στη Node.js

Με τη χρήση της ενσωματωμένης βιβλιοθήκης [crypto](#):

```
import crypto from 'crypto';

const input = "ευαίσθητες πληροφορίες";
const hash = crypto.createHash('sha256');
hash.update(input);
console.log(`${hash.digest('hex')}`);
// c0ea85d29735f34eb1537d19b6963c7c62ef217082c95ef120fa7911055c9608

const input2 = "ευαίσθητες πληροφορίες2";
const hash2 = crypto.createHash('sha256');
hash2.update(input2);
console.log(`${hash2.digest('hex')}`);
// acee6904f38ab5ddd6975042d031bebed78b590837a1390c9947b097fb18ebb7
```

- Οι χρήστες συχνά χρησιμοποιούν **συνήθη συνθηματικά**, π.χ.
'password', 'secret', '123456'

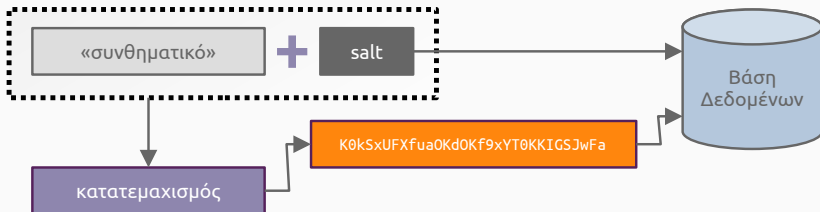
- Οι χρήστες συχνά χρησιμοποιούν **συνήθη συνθηματικά**, π.χ. 'password', 'secret', '123456'
 - αν στη βάση δεδομένων βρεθούν δύο ίδιες περιλήψεις ξέρουμε πως το συνθηματικό είναι κοινό → κάποιο από τα συνήθη συνθηματικά.

- Οι χρήστες συχνά χρησιμοποιούν **συνήθη συνθηματικά**, π.χ. 'password', 'secret', '123456'
 - αν στη βάση δεδομένων βρεθούν δύο ίδιες περιλήψεις ξέρουμε πως το συνθηματικό είναι κοινό → κάποιο από τα συνήθη συνθηματικά.
- Μπορεί κάποιος επιτεθέμενος να χρησιμοποιήσει έτοιμα “λεξικά” (rainbow table) με υπολογισμένα ενδιάμεσα αποτελέσματα από περιλήψεις που έχουν ήδη υπολογιστεί.

- Οι χρήστες συχνά χρησιμοποιούν **συνήθη συνθηματικά**, π.χ. 'password', 'secret', '123456'
 - αν στη βάση δεδομένων βρεθούν δύο ίδιες περιλήψεις ξέρουμε πως το συνθηματικό είναι κοινό → κάποιο από τα συνήθη συνθηματικά.
- Μπορεί κάποιος επιτεθέμενος να χρησιμοποιήσει έτοιμα “λεξικά” (rainbow table) με υπολογισμένα ενδιάμεσα αποτελέσματα από περιλήψεις που έχουν ήδη υπολογιστεί.
- Η συνάρτηση κατατεμαχισμού είναι πολύ γρήγορη και μπορούν να δοκιμαστούν πολλοί συνδυασμοί με μεγάλη ταχύτητα (brute force attack).

Κατατεμαχισμός με αλάτισμα

Η προσθήκη ενός **τυχαίου** αλφαριθμητικού (**salt**) κατά την παραγωγή της περίληψης είναι εύκολη και ισχυρή άμυνα σε τέτοιες επιθέσεις.



Σχήμα 11: Κατατεμαχισμός με αλάτι. Η περίληψη θα προκύψει από **συνθηματικό + τυχαίο αλάτι**, όπου το αλάτι είναι γνωστό στο σύστημα και διαφορετικό για κάθε συνθηματικό.

Ακόμη και αν δύο ή περισσότεροι χρήστες έχουν επιλέξει το ίδιο συνθηματικό, η τελική περίληψη θα είναι διαφορετική.

Ο αλγόριθμος κατατεμαχισμού bcrypt 1/4

Ο αλγόριθμος bcrypt θεωρείται σήμερα το πρότυπο των αλγορίθμων κατατεμαχισμού.

Σχεδιάστηκε από τους [Niels Provos](#) και [David Mazières](#) που βασίστηκαν στον [Blowfish](#): b από τον Blowfish και crypt από τη συνάρτηση hashing που χρησιμοποιείται από το σύστημα κωδικών χρηστών του UNIX.

Ο αλγόριθμος κατατεμαχισμού bcrypt 2/4

- Ο αλγόριθμος [bcrypt](#) είναι πολύ πιο ισχυρός από τον [MD5](#) ή το [SHA1](#) (που δεν είναι ασφαλείς),

Ο αλγόριθμος κατατεμαχισμού bcrypt 2/4

- Ο αλγόριθμος [bcrypt](#) είναι πολύ πιο ισχυρός από τον [MD5](#) ή το [SHA1](#) (που δεν είναι ασφαλείς),
 - περιλαμβάνει κύκλους “αλατίσματος” (salting) (προσθήκη χαρακτήρων), κάθε κύκλος αυξάνει τα βήματα υπολογισμού,

Ο αλγόριθμος κατατεμαχισμού bcrypt 2/4

- Ο αλγόριθμος [bcrypt](#) είναι πολύ πιο ισχυρός από τον [MD5](#) ή το [SHA1](#) (που δεν είναι ασφαλείς),
 - περιλαμβάνει κύκλους “αλατίσματος” (salting) (προσθήκη χαρακτήρων), κάθε κύκλος αυξάνει τα βήματα υπολογισμού,
 - *Γιατί θέλουμε ο υπολογισμός της περίληψης ενός συνθηματικού να μην είναι πολύ γρήγορος;*

Ο αλγόριθμος κατατεμαχισμού bcrypt 2/4

- Ο αλγόριθμος [bcrypt](#) είναι πολύ πιο ισχυρός από τον [MD5](#) ή το [SHA1](#) (που δεν είναι ασφαλείς),
 - περιλαμβάνει κύκλους “αλατίσματος” (salting) (προσθήκη χαρακτήρων), κάθε κύκλος αυξάνει τα βήματα υπολογισμού,
 - *Γιατί θέλουμε ο υπολογισμός της περίληψης ενός συνθηματικού να μην είναι πολύ γρήγορος;*
 - σήμερα θεωρείται επαρκής τιμή saltRounds = 10 (δηλ. 2^{10}), στο μέλλον η τιμή μπορεί να αυξηθεί.

Ο αλγόριθμος κατατεμαχισμού bcrypt 2/4

- Ο αλγόριθμος [bcrypt](#) είναι πολύ πιο ισχυρός από τον [MD5](#) ή το [SHA1](#) (που δεν είναι ασφαλείς),
 - περιλαμβάνει κύκλους “αλατίσματος” (salting) (προσθήκη χαρακτήρων), κάθε κύκλος αυξάνει τα βήματα υπολογισμού,
 - *Γιατί θέλουμε ο υπολογισμός της περίληψης ενός συνθηματικού να μην είναι πολύ γρήγορος;*
 - σήμερα θεωρείται επαρκής τιμή `saltRounds = 10` (δηλ. 2^{10}), στο μέλλον η τιμή μπορεί να αυξηθεί.
- Συνεπώς ο αλγόριθμος bcrypt προσαρμόζεται στις εξελίξεις της ταχύτητας υπολογισμών.

Ο αλγόριθμος κατατεμαχισμού bcrypt 3/4

```
import bcrypt from "bcrypt"
const myPassword = "πολύδύσκολονατομαντέψω"

for (let cost = 0; cost <= 19; cost++) {
  console.time(`cost: ${cost}, time`)
  bcrypt.hashSync(myPassword, cost)
  console.timeEnd(`cost: ${cost}, time`)
}
```

Αποτέλεσμα (Intel Core i7-8550U CPU):

...

cost: 16, time: 3.555s

cost: 17, time: 6.986s

cost: 18, time: 13.789s

cost: 19, time: 27.119s

...

Ο αλγόριθμος κατατεμαχισμού bcrypt 4/4

Στην περίληψη που παράγεται είναι ενσωματωμένο και το salt που χρησιμοποιήθηκε και δεν χρειάζεται να το αποθηκεύσουμε κάπου αλλού.

Η περίληψη που δημιουργείται με τον bcrypt έχει τη μορφή ↗

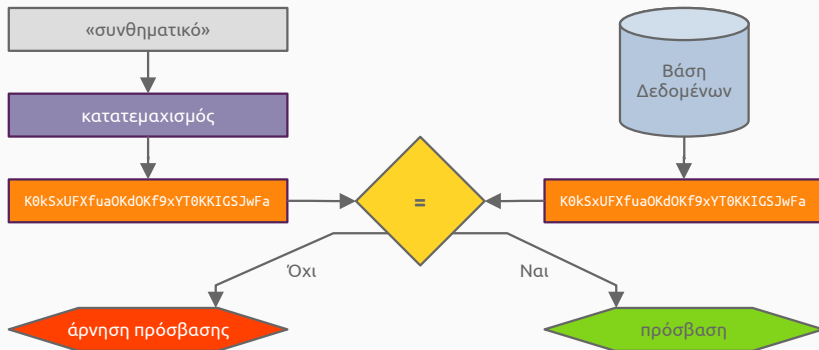
`$(algorithm)$(cost)$(salt)[hash]`

Για παράδειγμα:

`$2b$10$n0UIs5kJ7naTuTFkBy1veuK0kSxUFxfuaOKdOKf9xYT0K...`

			hash (24 Byte) =	K0kSxUFxfuaOKdOKf9xYT0K...
		salt (16 Byte) = n0UIs5kJ7naTuTFkBy1veu		
	cost => 10 = 2 ¹⁰ rounds			
algorithm => 2b = BCrypt				

Αυθεντικοποίηση χρηστών με κατατεμαχισμό



Σχήμα 12: Η αυθεντικοποίηση γίνεται συγκρίνοντας τις περιλήψεις

Controller διαχείρισης χρηστών

Η αρχική σελίδα “home” page, περιέχει επιλογή για GET /login, GET /register

```
app.get("/", (req, res) => {  
  res.render("home") });
```

Η σελίδα login, περιέχει φόρμα με υποβολή (POST) email, password

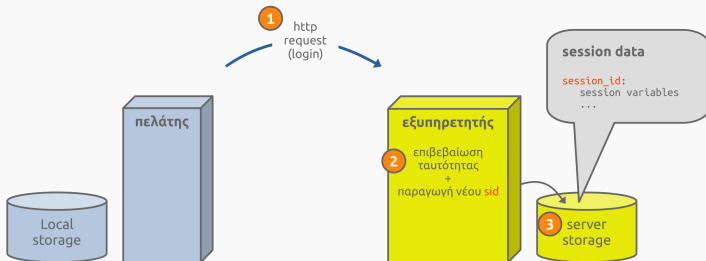
```
app.get("/login", (req, res) => {  
  res.render("login") });
```

```
// submit via POST the email, password, if success redirect to  
↪ "start"
```

```
app.post("/login", (req, res) => {  
  res.render("start");  
  else res.render("login");});
```

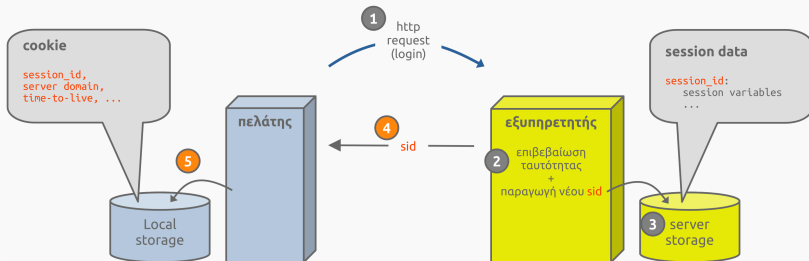
Διαχείριση συνεδρίας

Διαχείριση συνεδρίας 1



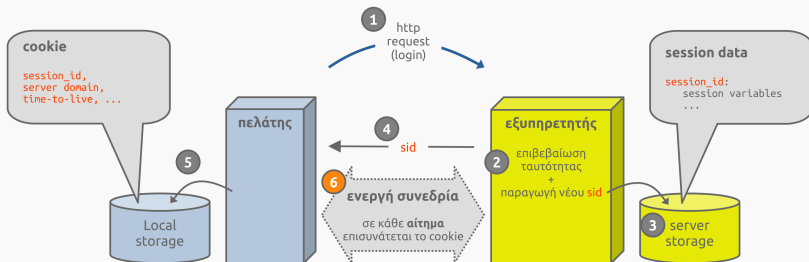
Σχήμα 13: Μετά την επιτυχή αυθεντικοποίηση παράγεται ένα μοναδικό **session id** (sid) και φυλάσσεται για περιορισμένο χρόνο στον εξυπηρετητή. Σε ένα **sid** μπορεί να αντιστοιχούν **μεταβλητές συνεδρίας**.

Διαχείριση συνεδρίας 2



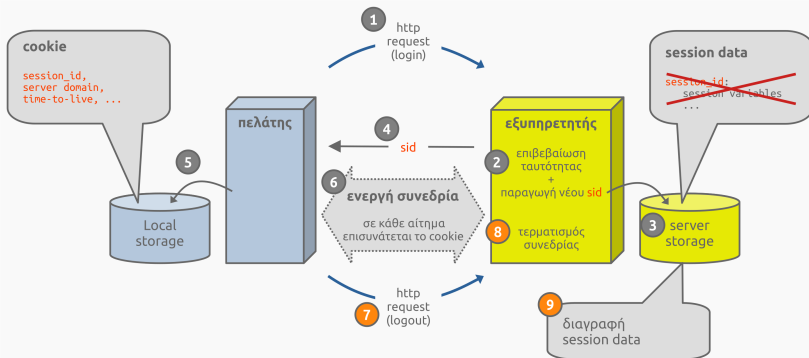
Σχήμα 14: Το session id επιστρέφεται στον πελάτη. Ο πελάτης το αποθηκεύει σε ένα **cookie**. Το cookie περιέχει, εκτός από το session id, και πληροφορίες σχετικά με domain στο οποίο ισχύει, το χρόνο ισχύος του, κ.ά.

Διαχείριση συνεδρίας 3



Σχήμα 15: Στο εξής, σε **κάθε αίτημα** του φυλλομετρητή στέλνεται και το cookie. Ο εξυπηρετητής θεωρεί πως τα αιτήματα που γίνονται σε κοντινά χρονικά διαστήματα (π.χ. 15 λεπτά) ανήκουν στην ίδια συνεδρία.

Διαχείριση συνεδρίας 4



Σχήμα 16: Στην αποσύνδεση (logout), ή όταν το διάστημα μεταξύ δύο διαδοχικών αιτημάτων ξεπεράσει κάποιο όριο, το session id διαγράφεται από τον εξυπηρετητή και πλέον το cookie δεν θα αναγνωρίζεται.

Express-session

```
import expSession from 'express-session'
const sessionConf = {
  secret: process.env.secret,
  cookie: {maxAge: 3600, sameSite: true}
  resave: false,
  saveUninitialized: false
};

app.use(expSession(sessionConf))
```

- **secret** : υποχρεωτικό, χρησιμοποιείται για να εξασφαλίσουμε πως τα Cookies που μας στέλνει ο πελάτης είναι όντως δικά μας.

Express-session

```
import expSession from 'express-session'
const sessionConf = {
  secret: process.env.secret,
  cookie: {maxAge: 3600, sameSite: true}
  resave: false,
  saveUninitialized: false
};

app.use(expSession(sessionConf))
```

- **resave** [↗](#), **saveUninitialized** [↗](#): έχουν προκαθορισμένες τιμές true αλλά αυτό θα αλλάξει σε μελλοντικές εκδόσεις.

Χρήση μεταβλητών περιβάλλοντος (dotenv)

```
npm install dotenv  
...  
import dotenv from 'dotenv/config'
```

στο αρχείο **.env** που βρίσκεται στο φάκελο του πρότζεκτ μας αποθηκεύουμε τις μεταβλητές που επιθυμούμε ως μεταβλητές περιβάλλοντας, πχ

KEY=miapolymegalilexipoykaneisdenmporeinafantastei

Έχουμε πρόσβαση στη μεταβλητή αυτή ως **process.env.KEY**



να καταχωρίσουμε το αρχείο **.env** στο **.gitignore**, πριν το ανεβάσουμε σε δημόσιο αποθετήριο, όπως το GitHub.

Ποιο είναι το session store στον εξυπηρετητή;

- Οι πληροφορίες για την κάθε συνεδρία (πχ το session id που στέλνουμε στον πελάτη και που το φυλάσσει σε μορφή cookie) αποθηκεύονται στον εξυπηρετητή από το [express-session](#) στη μνήμη, αλλά με τρόπο που **εσκεμμένα** δεν έχει υλοποιηθεί σωστά, ώστε να αναγκαστούμε να καταφύγουμε σε [άλλη 'αποθήκη'](#) για τις συνεδρίες.

Ποιο είναι το session store στον εξυπηρετητή;

- Οι πληροφορίες για την κάθε συνεδρία (πχ το session id που στέλνουμε στον πελάτη και που το φυλάσσει σε μορφή cookie) αποθηκεύονται στον εξυπηρετητή από το [express-session](#) στη μνήμη, αλλά με τρόπο που **εσκεμμένα** δεν έχει υλοποιηθεί σωστά, ώστε να αναγκαστούμε να καταφύγουμε σε [άλλη 'αποθήκη'](#) για τις συνεδρίες.
- Σε ένα σύστημα σε παραγωγική λειτουργία θα πρέπει να αποθηκευτούν σε μνήμη που να εκτείνεται, να είναι γρήγορη και να χρησιμοποιεί δευτερεύουσες μονάδες αποθήκευσης.

- Μια εύκολη εναλλακτική είναι το πακέτο [memorystore](#) που προσφέρει μια πιο πλήρη υλοποίηση της αποθήκευσης στη μνήμη του εξυπηρετητή.
- Υπάρχουν αρκετές διαφορετικές προτάσεις (πχ redis, mongo, sqlite3, κλπ), π.χ. μια συνήθης λύση είναι η χρήση του πακέτου connect-redis.

Χειρισμός σφαλμάτων στο Express.js

Χειρισμός σφαλμάτων στο Express.js

Για να χειριστούμε τα σφάλματα, μπορούμε να γράψουμε middleware, όπως συνήθως, με τη διαφορά ότι τα middleware αυτά έχουν τέσσερα ορίσματα αντί για τρία:

```
app.use((req, res, next) => {  
  throw new Error("σφάλμα!")  
})  
  
app.use((err, req, res, next) => {  
  console.error(err.stack)  
  res.status(500).send(err.message) //"σφαλμα!"  
})
```

Χειρισμός σφαλμάτων στο Express.js 2

Το Express.js ενσωματώνει έναν προκαθορισμένο χειριστή σφαλμάτων (default error handler).

Αυτός υπάρχει **πάντα** και είναι τελευταίος στην αλυσίδα επεξεργασίας.

Αν η τιμή της **μεταβλητής περιβάλλοντος**

NODE_ENV=production,

τότε αυτός ο προκαθορισμένος χειριστής σφαλμάτων δε θα δείξει στον φυλλομετρητή το stack trace.

Χειρισμός σφαλμάτων στο Express.js 3

Τα σφάλματα που επιστρέφονται από ασύγχρονες συναρτήσεις που καλούνται μέσα χειριστές διαδρομής ή middleware, πρέπει να τα μεταβιβάσουμε στη συνάρτηση **next()**, όπου το Express θα τα πιάσει και θα τα επεξεργαστεί. Για παράδειγμα:

```
app.get('/', (req, res, next) => {  
  try {  
    const data = await fsp.readFile('./file-does-not-exist', {  
      ↪ encoding: 'utf8' });  
    res.send(data)  
  }  
  catch (err) {  
    next(err) // μεταβίβαση του σφάλματος στο μηχανισμό χειρισμού  
      ↪ σφαλμάτων του Express.js  
  }  
})
```

Ασφαλής μεταφορά

Η ασφαλής αποθήκευση των διαπιστευτηρίων δεν έχει μεγάλη αξία αν η μεταφορά τους γίνεται μέσω HTTP, καθώς τα στοιχεία που στέλνει ο χρήστης δεν είναι κρυπτογραφημένα.

Η κρυπτογραφημένη εκδοχή του HTTP είναι το **HTTPS**.

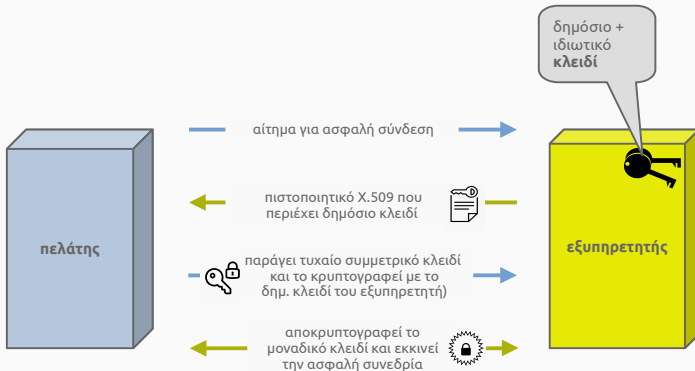
- η κρυπτογράφηση γίνεται με το κρυπτογραφικό πρωτόκολλο [TLS 1.3](#) (παλιότερα το SSL)

Η ασφαλής αποθήκευση των διαπιστευτηρίων δεν έχει μεγάλη αξία αν η μεταφορά τους γίνεται μέσω HTTP, καθώς τα στοιχεία που στέλνει ο χρήστης δεν είναι κρυπτογραφημένα.

Η κρυπτογραφημένη έκδοχή του HTTP είναι το **HTTPS**.

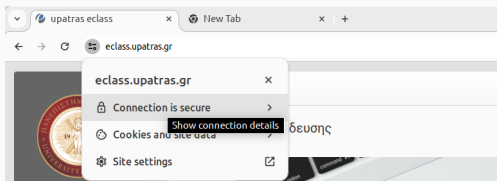
- η κρυπτογράφηση γίνεται με το κρυπτογραφικό πρωτόκολλο [TLS 1.3](#) (παλιότερα το SSL)
- είναι απαραίτητη η χρήση κάποιου πιστοποιητικού (από την πλευρά του εξυπηρετητή). -Ο εξυπηρετητής μπορεί να απαιτήσει συνδέσεις μόνο HTTPS με την κεφαλίδα απόκρισης [HTTP Strict-Transport-Security](#).

Ασφαλής μεταφορά με HTTP/TLS



Σχήμα 17: Η χρήση HTTPS απαιτεί ο εξυπηρετητής να διαθέτει ψηφιακό πιστοποιητικό με το οποίο να κρυπτογραφείται η επικοινωνία. Μόνο ο εξυπηρετητής χρειάζεται να έχει πιστοποιητικό.

Ασφαλής μεταφορά με HTTP/TLS



Σχήμα 18: Ένδειξη στον Google Chrome πως η σύνδεση είναι προστατευμένη.

Communicating the Dangers of Non-Secure HTTP [↗](#)

- **Αυθεντικοποίηση:** Κάθε μέρος μπορεί να επιβεβαιώσει την ταυτότητα του άλλου.
- **Κρυπτογράφηση:** Η επικοινωνία είναι προστατευμένη από τρίτους.
- **Ακεραιότητα:** Τα δεδομένα που μεταδίδονται δεν αλλοιώνονται.

Επίπεδα επικύρωσης πιστοποιητικών

Τρία επίπεδα, που αντιστοιχούν στο πόσο εξονοχυχιστικός είναι ο έλεγχος της ταυτότητας του κατόχου πριν εκδοθεί το πιστοποιητικό

- **Domain Validation (DV)**, επικύρωση πως ο κάτοχος του πιστοποιητικού είναι αυτός που διαχειρίζεται το domain.
- **Organization Validation (OV)**, επικύρωση πως ο κάτοχος του πιστοποιητικού είναι αυτός που διαχειρίζεται το domain και πως υπάρχει η νομική οντότητα του κατόχου (DV + *όνομα οργανισμού, ταχυδρομική διεύθυνση κλπ*).
- **Extended Validation (EV)**, επικυρώνει την νομική οντότητα του κατόχου του πιστοποιητικού με ανθρώπινο έλεγχο (DV + OV + *τηλ., χρονικό διάστημα ύπαρξης, νομική οντότητα κλπ*).

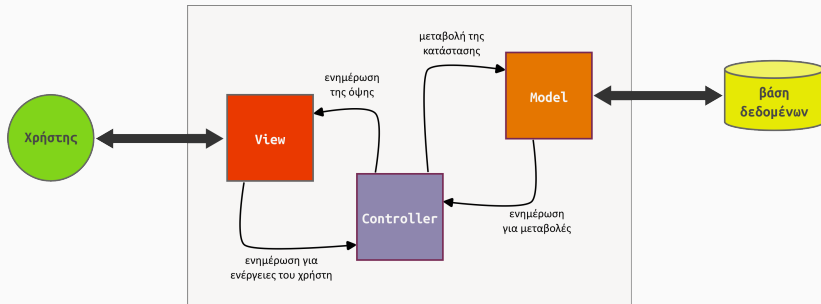
Πιστοποιητικά TLS (X.509)

- **Αρχή πιστοποίησης** (certification authority), επί πληρωμή
 - ο φυλλομετρητής τα εμπιστεύεται
 - πιστοποιείται **DV**, **OV** και **EV**
 - έχει κόστος
- **Let's Encrypt**, ισχύει για 3 μήνες, αυτόματη ανανέωση
 - ο φυλλομετρητής τα εμπιστεύεται
 - πιστοποιείται μόνο **DV**
 - δωρεάν, **συνίσταται** για τις περισσότερες περιπτώσεις
- **Self-signed certificate**, δεν έχει πιστοποίηση από κάποια αρχή πιστοποίησης
 - ο φυλλομετρητής δεν τα εμπιστεύεται, πρέπει να αποδεχτούμε το πιστοποιητικό με δική μας ενέργεια
 - πιστοποιείται **DV**
 - **δε συνίσταται**

Συγκρίνετε <https://www.digicert.com>, <https://hci.ece.upatras.gr/>, και το παράδειγμα 8-self-signed.

Task-list τελική έκδοση

Model-View-Controller



Passport



Το passport.js [↗](#) είναι ένα middleware για Express.js. Επιτρέπει τον έλεγχο ταυτότητας του χρήστη με ιδιαίτερα ευέλικτο τρόπο, χρησιμοποιώντας τον αλγόριθμο bcrypt [↗](#). Επίσης με χρήση sessions/cookies διατηρεί την πρόσβαση ενός χρήστη που έχει κάνει login στις σελίδες της εφαρμογής.



Επιτρέπει διάφορες στρατηγικές ελέγχου ταυτότητας του χρήστη μεταξύ των οποίων τοπικό έλεγχο ταυτότητας (passport-local) έλεγχο μέσω Facebook (passport-facebook) έλεγχος μέσω Google (passport-google), κλπ. Υποστηρίζονται πάνω από 500 τέτοιες στρατηγικές.